

Application E-Commerce Flutter avec Riverpod

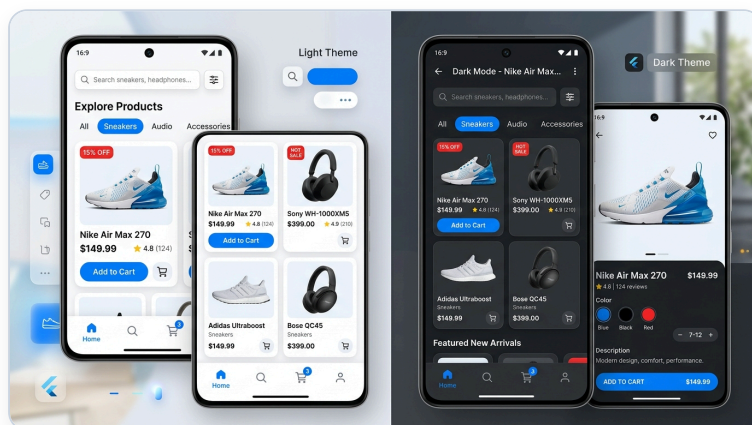
Comprendre l'Architecture, Répondre à Toutes les Questions du Cours et Recréer le Projet de Zéro

🎯 **Objectif** : Démystifier Flutter, Dart et Riverpod pour les débutants complets.

📱 **Projet** : ShopFlux — Application E-Commerce Multi-écrans Réactive.

🧠 **Compétences validées** : Gestion d'état avancée, Architecture en couches, Cycle asynchrone (`AsyncValue`), Persistance locale (`SharedPreferences`), Tests automatisés.

👤 **Niveau** : Accessible Débutant (0 expérience requise).



Aperçu de l'application ShopFlux développée dans le cadre de ce cours

1. Introduction : Qu'est-ce que Flutter et Riverpod ?

1.1. Comment fonctionne Flutter ?

Dans Flutter, **absolument tout est un Widget** : un texte, un bouton, une image, un espacement (`SizeBox`), et même la structure d'une page (`Scaffold`). Ces widgets s'emboîtent les uns dans les autres pour former ce qu'on appelle l'**Arbre des Widgets (Widget Tree)**.

1.2. Pourquoi le `setState` classique ne suffit-il pas ?

Quand vous débutez en Flutter, vous utilisez `setState()` dans un `StatefulWidget` . Cependant, cela pose deux problèmes majeurs dans une vraie application e-commerce :

- **Le problème du « Prop Drilling »** : Si vous ajoutez un article au panier depuis l'écran Catalogue, comment la barre de navigation ou l'écran Panier (qui se trouvent ailleurs dans l'arbre) sont-ils informés sans passer manuellement des dizaines de variables à travers tous les widgets intermédiaires ?
- **La reconstruction inutile** : `setState()` reconstruit tout le widget parent, ce qui dégrade les performances sur mobile.

💡 La solution : Riverpod

Riverpod est une solution de *Gestion d'État (State Management)* réactive. Il permet de stocker les données (le panier, les favoris, les produits) dans des conteneurs isolés appelés **Providers** situés en dehors des widgets. N'importe quel widget peut « écouter » uniquement la donnée dont il a besoin et se rafraîchir automatiquement dès qu'elle change.

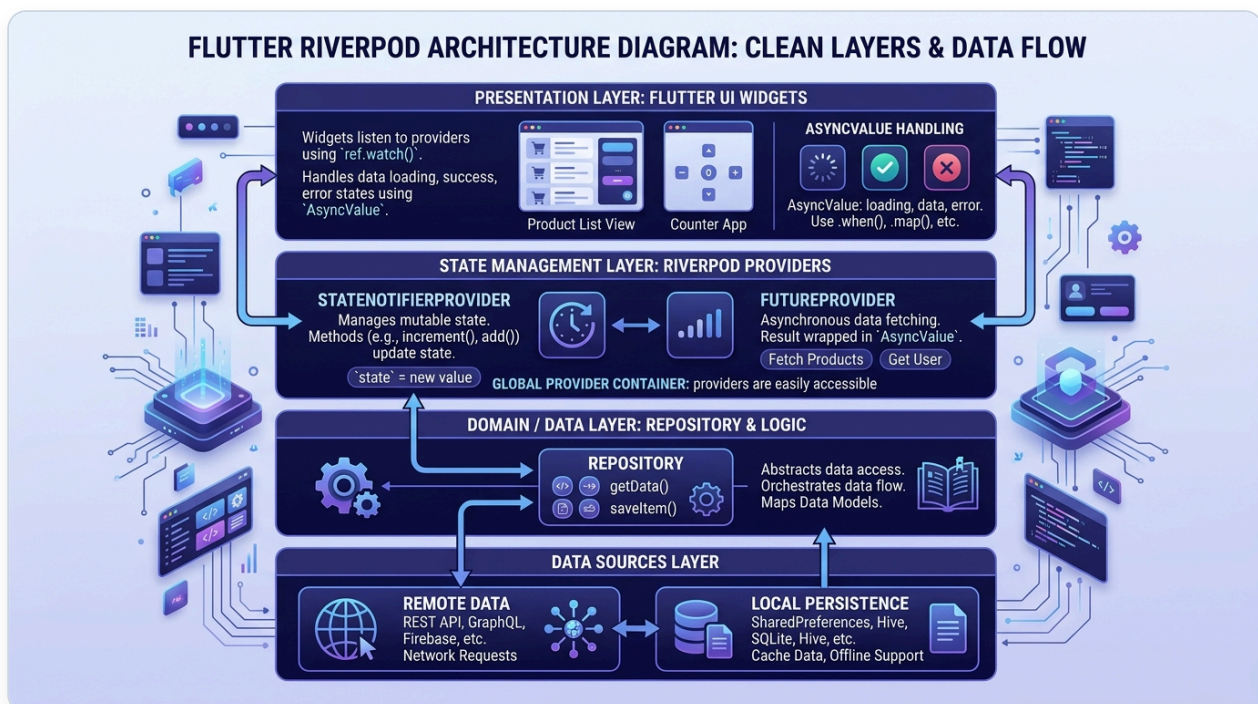


Schéma d'architecture réactive de notre application avec Riverpod

2. Les 3 Règles d'Or de Riverpod (À Connaître par Cœur)

Règle 1 : Le point de départ obligatoire — `ProviderScope`

Pour que Riverpod fonctionne, toute votre application doit être enveloppée dans un widget spécial nommé `ProviderScope` au tout début de votre fichier `main.dart`. C'est la « boîte magique » qui stocke l'état de tous vos providers.

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final prefs = await SharedPreferences.getInstance();

  runApp(
    ProviderScope(
      overrides: [
        sharedPreferencesProvider.overrideWithValue(prefs),
      ],
      child: const ShopFluxApp(),
    ),
  );
}
```

Règle 2 : Le Widget qui écoute — `ConsumerWidget` & `WidgetRef`

Pour qu'un écran ou un widget puisse lire un provider, il ne doit pas hériter de `StatelessWidget`, mais de `ConsumerWidget`. Sa méthode `build` reçoit un paramètre supplémentaire magique : `WidgetRef ref`.

```
// ❌ Ancien widget classique
class MyScreen extends StatelessWidget { ... }

// ✅ Widget compatible Riverpod
class MyScreen extends ConsumerWidget {
  const MyScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Vous pouvez maintenant utiliser 'ref' !
    final cart = ref.watch(cartProvider);
    return Text('Articles: ${cart.length}');
  }
}
```

Règle 3 : Le trio `ref.watch`, `ref.read` et `ref.listen`

Méthode	Quand l'utiliser ?	Exemple concret
<code>ref.watch(provider)</code>	Dans la méthode <code>build()</code> pour observer une donnée. Dès que la donnée change, le widget se	<code>final items = ref.watch(cartProvider);</code>

	redessine automatiquement.	
<code>ref.read(provider)</code>	Dans les événements/boutons (<code>onPressed</code> , <code>onTap</code>) pour exécuter une action ponctuelle sans s'abonner aux changements permanents.	<code>ref.read(cartProvider.notifier).addToCart(product);</code>
<code>ref.listen(provider, (prev, next))</code>	Pour déclencher des effets de bord (afficher un <code>SnackBar</code>, afficher une boîte de dialogue, changer de page).	<code>ref.listen(cartProvider, (prev, next) => showSnackBar());</code>

⚠ Erreur classique de débutant à éviter absolument :

N'utilisez **JAMAIS** `ref.read()` directement dans le corps de la méthode `build()` , car le widget ne sera pas notifié lors des mises à jour. Utilisez toujours `ref.watch()` dans `build()` !

3. Les Différents Types de Providers Utilisés

Type de Provider	Rôle et Cas d'Usage	Exemple dans notre Projet
FutureProvider	Gère une opération asynchrone (appel API, lecture de fichier JSON) et renvoie automatiquement un AsyncValue (Loading / Data / Error).	productsFutureProvider (charge le catalogue de produits avec latence réseau simulée).
StateNotifierProvider	Gère un état complexe modifiable via une classe StateNotifier contenant vos méthodes métier.	cartNotifierProvider (ajoute, supprime, incrémente les articles du panier).
Provider (Calculé / Dérivé)	Calcule une valeur en lecture seule à partir d'autres providers sans dupliquer les données.	cartSummaryProvider (calcule la TVA, la remise et le total à partir des articles du panier).
StateProvider	Gère une variable primitive simple (un booléen, une chaîne de recherche, un code promo).	simulateErrorProvider (active ou désactive la simulation d'erreur serveur).
Modificateur .family	Permet de passer un paramètre à un provider (comme l'ID d'un produit).	productDetailProvider(id) (récupère la fiche d'un produit spécifique).

3.1. Comment gérer le chargement et les erreurs avec **AsyncValue** ?

Quand vous chargez des données depuis Internet ou un fichier, l'application passe par 3 états :

1. **Chargement (Loading)** : On affiche un cercle de chargement (**CircularProgressIndicator**).
2. **Erreur (Error)** : Si le réseau échoue, on affiche un message clair avec un bouton « Réessayer ».
3. **Données reçues (Data)** : On affiche la liste des produits reçus.

```
// Dans notre code : AsyncValue.when gère les 3 états de façon élégante
final productsAsync = ref.watch(productsFutureProvider);

return productsAsync.when(
  data: (products) => ProductGrid(products: products),
  loading: () => const Center(child: CircularProgressIndicator()),
  error: (error, stack) => Center(
    child: Column(
      children: [
        Text('Erreur : $error'),
        ElevatedButton(
          onPressed: () => ref.invalidate(productsFutureProvider),
          child: const Text('Réessayer'),
        ),
      ],
    ),
  ),
);
```

4. L'Architecture du Projet : Organisation des Dossiers

Pour obtenir 100/100 dans un projet professionnel, le code doit être scrupuleusement rangé en couches étanches :

```
lib/
├── models/                # 1. LES MODÈLES DE DONNÉES (Classes pures Dart)
│   ├── product.dart      -> Représente un Produit (id, titre, prix, image...)
│   ├── cart_item.dart    -> Représente un Article du Panier (produit, quantité, variante)
│   ├── filter_state.dart -> Représente les critères de recherche et tri
│   └── user_profile.dart  -> Représente l'utilisateur et ses commandes
├── repositories/         # 2. LA COUCHE DE DONNÉES
│   ├── product_repository.dart -> Charge les données depuis assets/products.json
│   └── mock_products.dart   -> Catalogue de secours avec photos réalistes
├── services/             # 3. LES SERVICES DE PERSISTANCE LOCALE
│   └── favorites_storage.dart -> Sauvegarde les favoris dans SharedPreferences (disque)
├── providers/            # 4. LA LOGIQUE MÉTIER & GESTION D'ÉTAT RIVERPOD
│   ├── product_provider.dart -> Provider de produits et catégories
│   ├── cart_provider.dart    -> StateNotifieur du panier d'achat
│   ├── favorites_provider.dart -> StateNotifieur des favoris avec persistance
│   ├── filter_provider.dart  -> StateNotifieur des filtres et tri
│   ├── theme_provider.dart   -> StateNotifieur pour le Dark / Light Mode
│   └── providers.dart        -> Export centralisé de tous les providers
├── screens/              # 5. LES ÉCRANS PRINCIPAUX DE L'APPLICATION
│   ├── home_catalog_screen.dart -> Écran d'accueil avec recherche et grille
│   ├── product_detail_screen.dart -> Écran détail avec galerie photo et ajout panier
│   ├── cart_screen.dart      -> Écran panier avec code promo et commande
│   ├── favorites_screen.dart  -> Écran liste des favoris
│   ├── profile_screen.dart    -> Écran profil, commandes et tests développeur
│   └── main_navigation_screen.dart -> Barre de navigation inférieure avec badges animés
├── widgets/              # 6. COMPOSANTS RÉUTILISABLES
│   ├── product_card.dart     -> Carte individuelle de produit
│   ├── filter_bottom_sheet.dart -> Feuille modale de filtres et tri
│   ├── animated_cart_badge.dart -> Badge animé avec rebond lors d'un ajout
│   └── async_value_widget.dart -> Wrapper universel pour gérer AsyncValue
└── main.dart              # 7. POINT D'ENTRÉE DE L'APPLICATION
```

5. Guide Pratique Débutant : Refaire le Projet Étape par Étape

1 Initialiser le projet et les dépendances

Créer un projet Flutter et déclarer dans `pubspec.yaml` :

```
dependencies:
  flutter:
    sdk: flutter
  flutter_riverpod: ^3.4.2
  shared_preferences: ^2.5.5

flutter:
  uses-material-design: true
  assets:
    - assets/
```

2 Créer le modèle Produit immuable

Dans `lib/models/product.dart`, définir la classe avec `fromJson`, `toJson`, `copyWith`, et la surcharge de `operator ==` et `hashCode` pour que les tests puissent comparer deux produits par valeur.

3 Créer le CartNotifier (Gestion du Panier)

Dans `lib/providers/cart_provider.dart`, étendre `StateNotifier<List<CartItem>>` :

```
class CartNotifier extends StateNotifier<List<CartItem>> {
  CartNotifier() : super([]);

  void addToCart(Product product, [int quantity = 1]) {
    final index = state.indexWhere((item) => item.product.id == product.id);
    if (index != -1) {
      // Incrémenter la quantité d'un article existant
      final updated = List<CartItem>.from(state);
      updated[index] = updated[index].copyWith(quantity: updated[index].quantity +
quantity);
      state = updated;
    } else {
      // Ajouter un nouvel article
      state = [...state, CartItem(id: product.id, product: product, quantity: quantity)];
    }
  }

  void removeFromCart(String productId) {
    state = state.where((item) => item.product.id != productId).toList();
  }

  void incrementQuantity(String id) => updateQuantity(id, getItemQuantity(id) + 1);
  void decrementQuantity(String id) => updateQuantity(id, getItemQuantity(id) - 1);
}
```

4 Persister les Favoris sur le Disque

Dans `lib/services/favorites_storage.dart`, utiliser `SharedPreferences` :

```
class FavoritesStorage {
  final SharedPreferences _prefs;
  static const _key = 'user_favorite_product_ids';

  FavoritesStorage(this._prefs);

  Set<String> loadFavoriteIds() => _prefs.getStringList(_key)?.toSet() ?? {};

  Future<void> saveFavoriteIds(Set<String> ids) async {
    await _prefs.setStringList(_key, ids.toList());
  }
}
```

5 Construire l'UI avec `ref.watch`

Dans vos écrans, connectez les boutons avec `ref.read()` et observez les listes avec `ref.watch()`.

6. Foire Aux Questions (FAQ) : Répondre à Toutes les Questions du Cours

? Q1 : Quelle est la différence entre `ref.watch` et `ref.read` ?

Réponse : `ref.watch` est utilisé dans la méthode `build()` pour créer une souscription réactive : chaque fois que la valeur du provider change, le widget se reconstruit automatiquement. À l'inverse, `ref.read` est utilisé dans les gestionnaires d'événements (comme `onPressed`) pour lire la valeur ou appeler une méthode une seule fois sans s'abonner aux futures modifications.

? Q2 : Pourquoi utiliser `AsyncValue` au lieu de `FutureBuilder` ?

Réponse : `AsyncValue` de Riverpod encapsule proprement les états `data`, `loading` et `error` de manière typée et sécurisée. Contrairement à `FutureBuilder` qui peut relancer un appel réseau à chaque reconstruction du widget, `AsyncValue` garde les données en cache, gère le rafraîchissement (pull-to-refresh) et permet d'invalider l'état simplement avec `ref.invalidate()`.

? Q3 : Comment fonctionne la persistance locale avec `SharedPreferences` ?

Réponse : `SharedPreferences` écrit des paires clé-valeur (ici une liste d'identifiants de produits sous forme de `List<String>`) directement dans la mémoire non-volatile du smartphone. Lorsque l'application est fermée puis rouverte, le `FavoritesNotifier` lit ces identifiants au démarrage pour restaurer l'état exact de l'utilisateur.

? Q4 : Qu'est-ce qu'un Provider dérivé ? Donne un exemple.

Réponse : Un provider dérivé est un `Provider` qui écoute un ou plusieurs autres providers pour calculer une nouvelle valeur automatiquement. Exemple : `filteredProductsProvider` écoute à la fois la liste brute des produits (`productsFutureProvider`) et l'état des filtres (`filterNotifierProvider`) pour produire une liste triée et filtrée en direct.

? Q5 : Pourquoi surcharger `operator ==` et `hashCode` dans nos modèles ?

Réponse : Par défaut en Dart, deux instances d'objets distinctes avec les mêmes valeurs sont considérées comme différentes car elles pointent vers deux adresses mémoire distinctes. En surchargeant `==` et `hashCode`, nous permettons à Dart et aux tests de vérifier l'égalité *par valeur* (deux produits avec le même ID et le même prix sont considérés égaux).

? Q6 : Quel est le rôle de `ProviderScope` ?

Réponse : `ProviderScope` est le conteneur racine qui alloue et stocke l'état de tous les providers de l'arbre Flutter. Il permet également d'injecter des surcharges (*overrides*), ce qui est indispensable pour les tests unitaires et pour initialiser des services asynchrones comme `SharedPreferences`.

7. Lexique des Termes Essentiels

- **Widget** : Composant visuel ou structurel de base dans Flutter.
- **State (État)** : Ensemble des données dynamiques d'une application à un instant T (contenu du panier, thème actif, etc.).
- **StateNotifier** : Classe qui contient l'état et les fonctions pour le modifier de façon immuable.
- **Immuabilité** : Principe selon lequel un objet ne peut pas être altéré après sa création. Pour modifier un état, on en produit une nouvelle copie (`state = [...state, newItem]`).
- **Dependency Injection (DI)** : Mécanisme permettant de fournir à une classe ses dépendances externes sans qu'elle ait besoin de les instancier elle-même.

Document généré spécialement pour l'apprentissage et la validation des compétences Flutter & Riverpod.